

REFERENCE

CLI reference

📄 Copy page



Complete reference for Claude Code command-line interface, including commands and flags.

CLI commands

You can start sessions, pipe content, resume conversations, and manage updates with these commands:

Command	Description	Example
<code>claude</code>	Start interactive session	<code>claude</code>
<code>claude "query"</code>	Start interactive session with initial prompt	<code>claude "explain this project"</code>
<code>claude -p "query"</code>	Query via SDK, then exit	<code>claude -p "explain this function"</code>
<code>cat file claude -p "query"</code>	Process piped content	<code>cat logs.txt claude -p "explain"</code>
<code>claude -c</code>	Continue most recent conversation in current directory	<code>claude -c</code>
<code>claude -c -p "query"</code>	Continue via SDK	<code>claude -c -p "Check for type errors"</code>
<code>claude -r "<session>" "query"</code>	Resume session by ID or name	<code>claude -r "auth-refactor" "Finish this PR"</code>
<code>claude update</code>	Update to latest version	<code>claude update</code>

Microsoft Foundry. Requires `--config` pointing at a `_gateway.yaml`. Available in Claude Code v2.1.195 and later.

`claude install`
`[version]`

Install or reinstall the native binary. Accepts a version like `2.1.118`, or `stable` or `latest`. See [Install a specific version](#)

`claude install`
`stable`

`claude auth login`

Sign in to your Anthropic account. Use `--email` to pre-fill your email address, `--sso` to force SSO authentication, and `--console` to sign in with Anthropic Console for API usage billing instead of a Claude subscription

`claude auth login --console`

`claude auth`
`logout`

Log out from your Anthropic account

`claude auth logout`

`claude auth`
`status`

Show authentication status as JSON. Use `--text` for human-readable output. Exits with code 0 if logged in, 1 if not

`claude auth status`

`claude agents`

Open [agent view](#) to monitor and dispatch parallel background sessions. Use `--cwd <path>` to show only sessions started under that directory, or `--json` to print active sessions as a JSON array for scripting (`--json --all` also includes completed background sessions). Pass `--permission-mode`, `--model`, `--effort`, or `--agent` to set [defaults for dispatched sessions](#). Accepts `--settings`, `--add-dir`, `--plugin-dir`, and `--mcp-config` like the top-level `claude` command. Opening agent view requires an interactive terminal

`claude agents --json`

`claude attach`
`<id>`

Attach to a [background session](#) in this terminal

`claude attach`
`7c5dcf5d`

`claude auto-mode`
`defaults`

Print the built-in [auto mode](#) classifier rules as JSON. Use `claude auto-mode config` to see your effective config with settings applied. `--label <prefix>` prints only the rules whose label starts with that prefix, matched case-insensitively. Requires Claude Code v2.1.208 or later

`claude auto-mode`
`defaults --label`
`'Git Destructive'`

	Rules from managed settings or the <code>--settings</code> flag still apply. Requires Claude Code v2.1.212 or later. See Inspect the defaults and your effective config	
<code>claude daemon status</code>	Print the background-session supervisor's state, version, socket directory, and worker count for diagnostics. Exits 1 if the supervisor isn't running	<code>claude daemon status</code>
<code>claude daemon stop --any</code>	Stop the background-session supervisor and the sessions it hosts. Pass <code>--keep-workers</code> to leave background sessions running so the next supervisor reconnects to them. <code>--any</code> confirms stopping an on-demand supervisor, which is the default. Use this to recover from an unresponsive supervisor	<code>claude daemon stop --any --keep-workers</code>
<code>claude doctor</code>	Print read-only installation and settings diagnostics from the terminal without starting a session, including install health, settings-file validation errors, and Remote Control eligibility. For the in-session setup checkup that can also apply fixes, run /doctor	<code>claude doctor</code>
<code>claude import [codex gemini]</code>	Start an interactive session that runs /import to bring configuration from other coding agents into Claude Code. Accepts the same <code>--dry-run</code> and <code>--yes</code> options as the command. Not available on Amazon Bedrock, Google Cloud's Agent Platform, Microsoft Foundry, or Claude Platform on AWS. Also unavailable when you turn off feature-flag fetching . Requires Claude Code v2.1.213 or later	<code>claude import codex --dry-run</code>
<code>claude logs <id></code>	Print recent output from a background session	<code>claude logs 7c5dcf5d</code>
<code>claude mcp</code>	Configure Model Context Protocol (MCP) servers	See the Claude Code MCP documentation .

authorization URL instead of opening a browser, then paste the redirect URL back at the prompt. Requires Claude Code v2.1.186 or later. See [**Authenticate from the command line**](#)

`claude mcp logout`
`<name>`

Clear stored OAuth credentials for an MCP server. Requires Claude Code v2.1.186 or later

`claude mcp logout`
`sentry`

`claude plugin`

Manage Claude Code [**plugins**](#). Alias: `claude plugins`. See [**plugin reference**](#) for subcommands

`claude plugin`
`install code-`
`review@claude-`
`plugins-official`

`claude project`
`purge [path]`

Delete all local Claude Code state for a project: transcripts, task lists, debug logs, file-edit history, prompt history lines, and the project's entry in `~/.claude.json`. Omit `[path]` to pick from an interactive list. Flags: `--dry-run` to preview, `-y / --yes` to skip confirmation, `-i / --interactive` to confirm each item, `--all` for every project. See [**Clear local data**](#)

`claude project purge`
`~/work/repo --dry-`
`run`

`claude remote-`
`control`

Start a [**Remote Control**](#) server to control Claude Code from Claude.ai or the Claude app. Runs in server mode (no local interactive session). See [**Server mode flags**](#). After you stop the server, you can bring back the sessions it was serving. See [**Resume sessions after stopping the server**](#)

`claude remote-`
`control --name "My`
`Project"`

`claude respawn`
`<id>`

Restart a [**background session**](#), running or stopped, with its conversation intact. Use `--all` to restart every running session, e.g. to pick up an updated Claude Code binary

`claude respawn`
`7c5dcf5d`

`claude rm <id>`

Remove a [**background session**](#) from the list. The conversation transcript stays on your local machine, available through `claude --resume`

`claude rm 7c5dcf5d`

`setup` for a guided operator walkthrough, `claude self-hosted-runner doctor` to **diagnose a deployed runner**, and `claude self-hosted-runner orchestrator` to spawn **on-demand runners**. Requires Claude Code v2.1.224 or later

`claude setup-token`

Generate a long-lived OAuth token for CI and scripts. Prints the token to the terminal without saving it. Requires a Claude subscription. See **Generate a long-lived token**

`claude setup-token`

`claude stop <id>`

Stop a **background session**. Also accepts `claude kill`

`claude stop 7c5dcf5d`

`claude ultrareview [target]`

Run **ultrareview** non-interactively. Prints findings to stdout and exits 0 on success or 1 on failure. Use `--json` for the raw payload and `--timeout <minutes>` to override the 30-minute default. Use `--post` on a `github.com` pull request target to post the finished findings to the PR as one plain comment from your GitHub account. `-no-post` is the default. Requires Claude Code v2.1.227 or later. See **Post findings to the pull request**

`claude ultrareview 1234 --json`

If you mistype a subcommand, Claude Code suggests the closest match and exits without starting a session. For example, `claude udpate` prints `Did you mean claude update?` .

As of v2.1.199, `claude --dangerously-skip-permissions daemon <subcommand>` runs the `daemon` subcommand. Earlier versions treated `daemon <subcommand>` as the prompt for a new interactive session, so the subcommand never ran when the flag came first, a common setup when `claude` is aliased to include the flag. Only a leading `--dangerously-skip-permissions` or `--allow-dangerously-skip-permissions` routes to `daemon` this way; any other leading flag still starts an interactive session.

Flag	Description	Example
<code>--add-dir</code>	Add additional working directories for Claude to read and edit files. Grants file access; most <code>.claude/</code> configuration is <u>not discovered</u> from these directories. Validates each path exists as a directory. To persist these directories across sessions, set <code>permissions.additionalDirectories</code> in settings	<code>claude --add-dir ../apps ../lib</code>
<code>--advisor <model></code>	Enable the server-side <u>advisor tool</u> for this session with a model alias, <code>fable</code> , <code>opus</code> , or <code>sonnet</code> , or a full model ID. Takes precedence over the <code>advisorModel</code> setting for the session. <code>fable</code> requires <u>Table 5 access</u>	<code>claude --advisor opus</code>
<code>--agent</code>	Specify an agent for the current session (overrides the <code>agent</code> setting)	<code>claude --agent my-custom-agent</code>
<code>--agents</code>	Define custom subagents dynamically via JSON. Accepts the <u>fields listed for CLI-defined subagents</u>	<code>claude --agents '{"reviewer": {"description": "Reviews code", "prompt": "You are a code reviewer"}}'</code>
<code>--allow-dangerously-skip-permissions</code>	Add <code>bypassPermissions</code> to the <code>Shift+Tab</code> mode cycle without starting in it. Lets you begin in a different mode like <code>plan</code> and switch to <code>bypassPermissions</code> later. See <u>permission modes</u>	<code>claude --permission-mode plan --allow-dangerously-skip-permissions</code>
<code>--allowedTools</code> , <code>--allowed-tools</code>	Tools that execute without prompting for permission. See <u>permission rule syntax</u> for pattern matching. To restrict which tools are available, use <code>--tools</code> instead. If you name one of the <u>task-tracking tools</u> here, Claude Code also opts the session in	<code>"Bash(git log *)"</code> <code>"Bash(git diff *)"</code> <code>"Read"</code>

	applies in non-interactive mode with <code>-p</code> . Requires Claude Code v2.1.205 or later	<code>answer" "query"</code>
<code>--append-system-prompt</code>	Append custom text to the end of the default system prompt	<code>claude --append-system-prompt "Always use TypeScript"</code>
<code>--append-system-prompt-file</code>	Load additional system prompt text from a file and append to the default prompt	<code>claude --append-system-prompt-file ./extra-rules.txt</code>
<code>--autocompact</code> <code><auto tokens></code>	Set the <u>auto-compact window</u> for this session without changing your saved settings. Accepts the same values as <code>/autocompact</code> ; that section covers the value forms and what overrides the flag. Requires Claude Code v2.1.221 or later	<code>claude --autocompact 500k</code>
<code>--ax-screen-reader</code>	Render screen-reader friendly output: flat text without decorative borders or animations. Forces the classic renderer, so the <code>tui</code> setting has no effect; attached <u>background sessions</u> still render fullscreen. Takes precedence over <code>CLAUDE_AX_SCREEN_READER</code> and the <code>axScreenReader</code> setting. Requires Claude Code v2.1.181 or later	<code>claude --ax-screen-reader</code>
<code>--bare</code>	Minimal mode: skip auto-discovery of hooks, skills, custom commands, subagents, plugins, MCP servers, auto memory, and CLAUDE.md so scripted calls start faster. Skills in a directory you pass with <code>--add-dir</code> still load. Claude has access to Bash, file read, and file edit tools. Sets <code>CLAUDE_CODE_SIMPLE</code> . See <u>bare mode</u>	<code>claude --bare -p "query"</code>
<code>--betas</code>	Beta headers to include in API requests (API key users only)	<code>claude --betas interleaved-thinking</code>

job instead of a Claude session, or with `--agent` to run a specific subagent. Cannot be combined with `-p` / `--print`; see the [error reference](#)

`--channels`

(Research preview) MCP servers whose **channel** notifications Claude should listen for in this session. Space-separated list of `plugin:<name>@<marketplace>` entries. Requires Anthropic authentication through `claude.ai` or a Console API key

```
claude --channels
plugin:my-
notifier@my-
marketplace
```

`--chrome`

Enable **Chrome browser integration** for web automation and testing

```
claude --chrome
```

`--cloud`

With a task description, create a new **web session** on `claude.ai`. With a session ID (`session_...` or `cse_...`) or a `claude.ai/code` URL, queue a message into that existing session instead, with `-p`. See [send a follow-up message](#).

```
claude --cloud "Fix
the login bug"
```

`--continue` , `-c`

Load the most recent conversation in the current directory, skipping **background sessions, sessions created with `claude -p` or the Agent SDK, and sessions whose first prompt was `/loop`**. `claude -p --continue` includes `-p`, SDK, and `/loop` sessions. Includes sessions that added this directory with `/add-dir`

```
claude --continue
```

`--dangerously-load-development-channels`

Enable **channels** that are not on the approved allowlist, for local development. Accepts `plugin:<name>@<marketplace>` and `server:<name>` entries. Prompts for confirmation

```
claude --dangerously-
load-development-
channels
server:webhook
```

`--dangerously-skip-permissions`

Skip permission prompts. Equivalent to `--permission-mode bypassPermissions`. See [permission modes](#) for what this does and does not skip. For sessions started with `--bg`, the mode **persists when the supervisor restarts the session**

```
claude --dangerously-
skip-permissions
```

mode without filtering

`--debug-file`
`<path>`

Write debug logs to a specific file path. Implicitly enables debug mode. Takes precedence over `CLAUDE_CODE_DEBUG_LOGS_DIR`

```
claude --debug-file  
/tmp/claude-  
debug.log
```

`--disable-slash-
commands`

Disable all skills and commands for this session

```
claude --disable-  
slash-commands
```

`--disallowedTools` ,
`--disallowed-tools`

Deny rules. A bare tool name removes the matching tools from Claude's context: `"Edit"` removes `Edit`, `"*"` removes every tool, and `"mcp_*"` removes every MCP tool. A scoped rule such as `Bash(rm *)` leaves the tool available and denies only matching calls. A rule naming `EndConversation` can't remove it while any other tool remains

```
"Bash(git log *)" "  
"Bash(git diff *)" "  
"Edit"
```

`--effort`

Set the **effort level** for the current session. Options: `low` , `medium` , `high` , `xhigh` , `max` , or `ultracode` . Available levels depend on the model. `ultracode` starts the session at `xhigh` effort with **ultracode** turned on, and requires Claude Code v2.1.203 or later. Overrides the `effortLevel` setting for this session and does not persist

```
claude --effort high
```

`--enable-auto-mode`

Removed in v2.1.111. Auto mode is now in the `Shift+Tab` cycle by default; use `--permission-mode auto` to start in it

```
claude --permission-  
mode auto
```

`--environment`
`<environment-id>`

Create a new cloud session that runs on the **self-hosted environment** with the given ID. Environment IDs start with `ccpool_`. See `--environment` **dispatch behavior** for dispatch behavior and the flag combinations it rejects. Requires Claude Code v2.1.224 or later

```
claude -p "Fix the  
login bug" --  
environment  
ccpool_abc123
```

different users and machines running the same task. Only applies with the default system prompt; ignored when `--system-prompt` or `--system-prompt-file` is set. Use with `-p` for scripted, multi-user workloads

`--exec`

Run a shell command as a PTY-backed background job instead of starting a Claude session. Use with `--bg` to launch from the shell

```
claude --bg --exec  
'pytest -x'
```

`--fallback-model`

Enable automatic fallback to the specified model(s) when the primary model is overloaded or not available, for example a retired model. Accepts a comma-separated list tried in order. See [Fallback model chains](#). To persist a chain across sessions, use the `fallbackModel` [setting](#), which this flag overrides

```
claude --fallback-  
model sonnet,haiku
```

`--fork-session`

When resuming, create a new session ID instead of reusing the original (use with `--resume` or `--continue`)

```
claude --resume  
abc123 --fork-  
session
```

`--forward-subagent-
text`

Emit [subagent](#) text and thinking blocks in the output stream as `assistant` and `user` messages with `parent_tool_use_id` set, so you can reconstruct each subagent's transcript. Without this flag, Claude Code emits only subagent `tool_use` and `tool_result` blocks. Requires `--print` and `--output-format stream-json` . Claude Code also forwards messages from [nested subagents](#), setting `parent_tool_use_id` to the ID of the Agent tool call that spawned each one; this requires Claude Code v2.1.219 or later. The `CLAUDE_CODE_FORWARD_SUBAGENT_TEXT` environment variable enables the same behavior. Requires Claude Code v2.1.211 or later

```
claude -p --output-  
format stream-json --  
verbose --forward-  
subagent-text  
"query"
```

	request URL. Sessions are linked automatically when Claude creates the pull request	
<code>--ide</code>	Automatically connect to IDE on startup if exactly one valid IDE is available	<code>claude --ide</code>
<code>--init</code>	Run Setup hooks with the <code>init</code> matcher before the session (print mode only)	<code>claude -p --init "query"</code>
<code>--init-only</code>	Run Setup and <code>SessionStart</code> hooks, then exit without starting a conversation	<code>claude --init-only</code>
<code>--include-hook-events</code>	Include hook lifecycle events in the output stream. <code>SessionStart</code> and <code>Setup</code> hook events are always included and don't need this flag. Some hook events, such as <code>Notification</code> , <code>SessionEnd</code> , <code>PreCompact</code> , and <code>PostCompact</code> , never produce a <code>hook_started</code> event, even with this flag. For those events, Claude Code still emits <code>hook_progress</code> while a command hook that runs for more than a second produces output, and emits <code>hook_response</code> only when a hook that runs in the background finishes. Requires <code>--output-format stream-json</code>	<code>claude -p --output-format stream-json --verbose --include-hook-events "query"</code>
<code>--include-partial-messages</code>	Include partial streaming events in output. Requires <code>--print</code> and <code>--output-format stream-json</code>	<code>claude -p --output-format stream-json --verbose --include-partial-messages "query"</code>
<code>--input-format</code>	Specify input format for print mode (options: <code>text</code> , <code>stream-json</code>)	<code>claude -p --output-format json --input-format stream-json</code>
<code>--json-schema</code>	Get validated JSON output matching a JSON Schema after the agent completes its workflow (print mode only). See structured outputs . Claude Code exits with an error on an invalid schema and accepts the <code>format</code> keyword as an annotation without client-side validation	<code>claude -p --json-schema '{"type":"object","properties":{...}}' "query"</code>
<code>--maintenance</code>	Run Setup hooks with the <code>maintenance</code> matcher before the session (print mode only)	<code>claude -p --maintenance "query"</code>

fails with `Budget limit reached`, and Claude Code stops background subagents that are still running; the cap-enforcement behaviors require Claude Code v2.1.217 or later

`--max-turns`

Limit the number of agentic turns (print mode only). Exits with an error when the limit is reached. No limit by default. With `--input-format stream-json`, a message still queued when the limit ends a turn stays queued and starts a new turn with its own limit

```
claude -p --max-turns
3 "query"
```

`--mcp-config`

Load MCP servers from JSON files or strings (space-separated). When you pass this flag with `-p`, Claude Code waits for still-pending servers to connect before running the first turn, up to the `MCP_TIMEOUT` startup timeout, 30 seconds by default; a server with a **cached tool list** skips the wait and connects on first use. The wait requires Claude Code v2.1.221 or later

```
claude --mcp-config
./mcp.json
```

`--model`

Sets the model for the current session with an alias for the latest model (`sonnet`, `opus`, `haiku`, or `fable`) or a model's full name. Overrides the `model` setting and `ANTHROPIC_MODEL`

```
claude --model
claude-sonnet-5
```

`--name`, `-n`

Set a display name for the session, shown in `/resume` and the terminal title. You can resume a named session with `claude --resume <name>`. In an interactive session, if another live session on this machine already uses the name, Claude Code applies **a variant of it** instead.

```
claude -n "my-
feature-work"
```

`/rename` changes the name mid-session and also shows it on the prompt bar

`--no-chrome`

Disable **Chrome browser integration** for this session

```
claude --no-chrome
```

	environment variable does the same in any mode	
<code>--output-format</code>	Specify output format for print mode (options: <code>text</code> , <code>json</code> , <code>stream-json</code>)	<code>claude -p "query" --output-format json</code>
<code>--permission-mode</code>	Begin in a specified <u>permission mode</u> . Accepts <code>default</code> , <code>acceptEdits</code> , <code>plan</code> , <code>auto</code> , <code>dontAsk</code> , <code>bypassPermissions</code> , or <code>manual</code> as an alias for <code>default</code> . The <code>manual</code> alias selects the permission mode the UI labels Manual and requires Claude Code v2.1.200 or later; <code>claude --help</code> lists it in place of <code>default</code> , and both values work. Overrides <code>defaultMode</code> from settings files. Without this flag or <code>--dangerously-skip-permissions</code> , a new session starts in the permission mode described in <u>which permission mode a session starts in</u> . For <code>-p</code> , that's <code>default</code> when nothing is configured	<code>claude --permission-mode plan</code>
<code>--permission-prompt-tool</code>	Specify an MCP tool to handle permission prompts in non-interactive mode. Claude Code waits for that tool's MCP server to connect before running the first turn, up to the <code>MCP_TIMEOUT</code> startup timeout, 30 seconds by default. The prompt tool can't approve an MCP tool marked as <u>requiring user interaction</u>: Claude Code converts an <code>allow</code> result for one to a deny. This restriction requires Claude Code v2.1.199 or later	<code>claude -p --permission-prompt-tool mcp_auth_tool "query"</code>
<code>--plugin-dir</code>	Load a plugin from a directory or <code>.zip</code> archive for this session only. Each flag takes one path. Repeat the flag for multiple plugins: <code>--plugin-dir A --plugin-dir B.zip</code>	<code>claude --plugin-dir ./my-plugin</code>
<code>--plugin-url</code>	Fetch a plugin <code>.zip</code> archive from a URL for this session only. Repeat the flag for multiple plugins, or pass space-separated URLs in a single quoted value	<code>claude --plugin-url https://example.com/plugin.zip</code>

<code>--prompt-suggestions</code>	Emit a <code>prompt_suggestion</code> message with a predicted next user prompt after each turn that generates one; very short conversations can produce none. Requires <code>--print</code> , <code>--output-format stream-json</code> , and <code>--verbose</code> . See Prompt suggestions	<code>claude -p --prompt-suggestions --output-format stream-json --verbose "query"</code>
<code>--ref <branch></code>	With <code>--environment</code> , base the new session's checkout on a named ref instead of local <code>HEAD</code>	<code>claude -p "Run the smoke test" --environment ccpool_abc123 --ref main</code>
<code>--remote</code>	Deprecated alias for <code>--cloud</code> , including the existing-session form	<code>claude --remote "Fix the login bug"</code>
<code>--remote-control</code> , <code>--rc</code>	Start an interactive session with Remote Control enabled so you can also control it from claude.ai or the Claude app. Optionally pass a name for the session	<code>claude --remote-control "My Project"</code>
<code>--remote-control-session-name-prefix <prefix></code>	Prefix for auto-generated Remote Control session names when no explicit name is set. Defaults to your machine's hostname, producing names like <code>myhost-graceful-unicorn</code> . Set <code>CLAUDE_REMOTE_CONTROL_SESSION_NAME_PREFIX</code> for the same effect	<code>claude remote-control --remote-control-session-name-prefix dev-box</code>
<code>--replay-user-messages</code>	Re-emit user messages from stdin back on stdout for acknowledgment. Requires <code>--input-format stream-json</code> and <code>--output-format stream-json</code>	<code>claude -p --input-format stream-json --output-format stream-json --verbose --replay-user-messages</code>

project settings. Claude Code removes the built-in tools that run commands or code, and WebFetch, unless you name them individually in `--tools`, not through its `default` preset. It also confines the built-in file tools to the **working directories**, loads only **managed settings** and `--settings`, and refuses **bypassPermissions**. Requires Claude Code v2.1.248 or later

`--resume`, `-r`

Resume a specific session by ID or name, or show an interactive picker to choose a session. The picker and name search include sessions that added this directory with `/add-dir`. When you pass a session ID, Claude Code searches the current project directory and its git worktrees, then every other project on this machine. Before v2.1.223, the ID search covered only the current project directory and its git worktrees. **Background sessions** appear in the picker marked with `bg`

```
claude --resume auth-  
refactor
```

`--safe-mode`

Start with all customizations disabled to troubleshoot a broken configuration: CLAUDE.md, skills, plugins, hooks, MCP servers, custom commands and agents, output styles, workflows, custom themes, custom keybindings, status line and file-suggestion commands, LSP servers, and auto memory do not load. Authentication, model selection, built-in tools, and permissions work normally, which differs from `--bare`. Managed settings policy still applies, including policy-configured hooks, status line, and file-suggestion commands; managed plugins, managed skills, managed CLAUDE.md, and policy-configured MCP servers do not. Useful for checking whether a customization is what triggers **automatic model fallback**. Sets `CLAUDE_CODE_SAFE_MODE`

```
claude --safe-mode
```

`--session-id`

Use a specific session ID for the conversation (must be a valid UUID)

```
claude --session-id  
"550e8400-e29b-41d4-  
a716-446655440000"
```

	string. Values you set here override the same keys in your <code>settings.json</code> files for this session. Keys you omit keep their file-based values. The file must be a regular file no larger than 2 MiB. See settings precedence	<code>./settings.json</code>
<code>--strict-mcp-config</code>	Only use MCP servers from <code>--mcp-config</code> , ignoring all other MCP configurations. See Exclusive control with managed-mcp.json for what the flag does under a managed MCP file	<code>claude --strict-mcp-config --mcp-config ./mcp.json</code>
<code>--system-prompt</code>	Replace the entire system prompt with custom text	<code>claude --system-prompt "You are a Python expert"</code>
<code>--system-prompt-file</code>	Load system prompt from a file, replacing the default prompt	<code>claude --system-prompt-file ./custom-prompt.txt</code>
<code>--teleport</code>	Resume a web session in your local terminal	<code>claude --teleport</code>
<code>--teammate-mode</code>	Set how agent team teammates display: <code>in-process</code> (default), <code>auto</code> , <code>tmux</code> , or <code>iterm2</code> (added in v2.1.186). Overrides the <code>teammateMode</code> setting for this session. See Choose a display mode	<code>claude --teammate-mode auto</code>
<code>--tmux</code>	Create a tmux session for the worktree. Requires <code>--worktree</code> . Uses iTerm2 native panes when available; pass <code>--tmux=classic</code> for traditional tmux	<code>claude -w feature-auth --tmux</code>
<code>--tools</code>	Restrict which built-in tools Claude can use. Use <code>""</code> to disable all, <code>"default"</code> for all, or tool names like <code>"Bash,Edit,Read"</code> . If you name one of the task-tracking tools here, Claude Code also opts the session in. The flag doesn't affect MCP tools; to deny those too, use <code>--disallowedTools "mcp__*"</code> . A list that omits <code>EndConversation</code> doesn't remove it; <code>""</code> removes it only when no MCP tools remain	<code>claude --tools "Bash,Edit,Read"</code>
<code>--verbose</code>	Enable verbose logging, shows full turn-by-turn output. Overrides the <code>viewMode</code> setting for this session	<code>claude --verbose</code>

don't give a name, Claude Code generates one.
Pass `#<number>`, a GitHub pull request URL, or
a GitLab merge request URL to **fetch that PR or
MR from origin and branch the worktree
from it**. Branching from a GitLab merge request
requires Claude Code v2.1.233 or later

System prompt flags

Claude Code provides four flags for customizing the system prompt. All four work in both interactive and non-interactive modes.

Flag	Behavior	Example
<code>--system-prompt</code>	Replaces the entire default prompt	<code>claude --system-prompt "You are a Python expert"</code>
<code>--system-prompt-file</code>	Replaces with file contents	<code>claude --system-prompt-file ./prompts/review.txt</code>
<code>--append-system-prompt</code>	Appends to the default prompt	<code>claude --append-system-prompt "Always use TypeScript"</code>
<code>--append-system-prompt-file</code>	Appends file contents to the default prompt	<code>claude --append-system-prompt-file ./style-rules.txt</code>

`--system-prompt` and `--system-prompt-file` are mutually exclusive. The append flags can be combined with either replacement flag.

Choose based on whether Claude Code's default identity still fits your task. Use an append flag when Claude should remain a coding assistant that also follows your extra rules: per-invocation instructions, output formatting, or domain context for a `-p` script. Appending preserves the default tool guidance, safety instructions, and coding conventions, so you only supply what differs. Use a replacement flag when the surface, identity, or permission model differs from Claude Code's, like a non-coding agent in a pipeline that no human watches. Replacing drops all of the default prompt, including tool

between and share across a project, use [output styles](#). For project conventions Claude should always follow, use [CLAUDE.md](#). The [Agent SDK guide on system prompts](#) covers the same decision in more depth.

See also

- [Chrome extension](#) - Browser automation and web testing
- [Interactive mode](#) - Shortcuts, input modes, and interactive features
- [Quickstart guide](#) - Getting started with Claude Code
- [Common workflows](#) - Advanced workflows and patterns
- [Settings](#) - Configuration options
- [Agent SDK documentation](#) - Programmatic usage and integrations

Was this page helpful?

 Yes

 No



Company

[Anthropic](#)

[Careers](#)

[Economic Futures](#)

[Research](#)

[News](#)

[Trust center](#)

[Transparency](#)

Learn

[Courses](#)

[MCP connectors](#)

[Customer stories](#)

[Engineering blog](#)

[Events](#)

[Powered by Claude](#)

[Service partners](#)

[Startups program](#)

Help and security

[Availability](#)

[Status](#)

[Support center](#)

Terms and policies

[Privacy choices](#)

[Privacy policy](#)

[Disclosure policy](#)

[Usage policy](#)

[Commercial terms](#)

[Consumer terms](#)